Key Cooperative Attribute-Based Encryption

Luqi Huang[1,✉] Willy Susilo[1] Guomin Yang[2] and Fuchun Guo[1]

[1] Institute of Cybersecurity and Cryptology, School of Computing and Information Technology, University of Wollongong, Wollongong, Australia
lh852@uowmail.edu.au, wsusilo@uow.edu.au, fuchun@uow.edu.au
[2] School of Computing and Information Systems, Singapore Management University, Singapore
gmyang@smu.edu.sg

Abstract. Attribute-based encryption (ABE) is an important technology in building access control systems with precise control and scalability. In an ABE system, there exists a private key generator (PKG) that issues all private keys. The PKG has a significant drawback referred to as the huge key management burden in large-scale user systems. To overcome this limitation, we propose a more flexible system that offers users the choice to utilize decryption keys either from the PKG or from trusted users to decrypt the ciphertext, reducing the workload of the PKG. Unfortunately, users are restricted to only receiving private keys from the PKG in most ABE schemes. Thus, our system ABE extends the ability of trusted users to generate and distribute decryption keys. Furthermore, decryption keys from trusted users possess equivalent decryption with a private key from the PKG when satisfying the cooperative access policy set by the encryptor. We present a key cooperative ABE scheme, defining the concept of key cooperative ABE for the first time.

Keywords: Attribute-based encryption · Key management · Key cooperative · Trust delegation.

1 Introduction

ABE plays a key role in realizing access control systems with fine granularity and scalability. It was proposed [24] by Sahai and Waters in 2005. As a promising cryptographic primitive, it has successfully attracted considerable research efforts and comes in two categories: Ciphertext-Policy ABE (CP-ABE) and Key-Policy ABE (KP-ABE). In a CP-ABE system, a party encrypting data can specify access to the data as a boolean formula over a set of attributes. A user will be able to decrypt a ciphertext if the attributes associated with their private key satisfy the boolean formula ascribed to the ciphertext.

In an ABE system, private keys were distributed by the PKG that would need to be in a position to verify all the attributes it issued for each user in the system. This approach imposes two key management challenges: key escrow and a huge key management burden. To reduce the workload of the PKG, we

propose a more flexible ABE system in terms of generating and distributing decryption keys. Specifically, users within this system can obtain decryption keys either from the PKG or from trusted users.

The common strategy for managing a substantial workload of the PKG is distributing its authority to other attribute authorities. One such approach is Hierarchical ABE (HABE) [27], which enables hierarchical access control by introducing hierarchical attribute authorities. The PKG is only responsible for the generation and distribution of system parameters and domain keys. Similarly, multi-authority ABE [5] introduced parallel attribute authorities to decentralize the PKG's power. These authorities monitor different sets of attributes and issue corresponding private keys to users. However, both HABE and multi-authority ABE still require users to heavily rely on these attribute authorities. Furthermore, the introduction of additional authorities resulted in increased storage costs and communication overheads.

Recently, registered ABE [10] allows users to generate private keys on their own. A key curator (KC) maintains no secrets and replaces the PKG to aggregate public keys from registered users into a master public key and generates helper decryption keys for users. However, removing the central authority (PKG) with secrets results in significant additional updating costs. Since the master public key of the registered ABE scheme changes whenever new users join the system, users must periodically refresh their helper decryption over the lifetime of the system. This can be particularly challenging in scenarios with a large number of users who frequently join and leave the system. As a result, registered ABE is limited to applications within fixed, small-scale user systems.

Different from the decentralization of PKG's power, online/offline ABE [11] divides the private key generation computation into two phases. However, the PKG remains responsible for generating all private keys, even when operating as an online server within this framework. Besides the above ABE schemes, some classic ABE [2, 9, 28] support one-to-one key delegation. Nevertheless, the scope for key delegation among users is restricted to a small number of individuals in the system.

In the Pretty Good Privacy (PGP) encryption program [22] and GNU Privacy Guard (GNU PG) software suite [1] for secure communication, a web of trust model allows the responsibility for validating public keys is delegated to people a user trusts. In this way, it avoids relying exclusively on a certificate authority and communicating with unknown parties. Inspired by the trust delegation of the decentralized web of trust model, our system considers delegating the generation and distribution of decryption keys to trusted users for reducing the huge workload of the PKG in an ABE system. Therefore, we propose a new ABE system, Key Cooperative ABE.

1.1 Our Contribution

In this paper, we present a novel ABE called Key Cooperative ABE (KC-ABE), aiming at the problem of huge workload for the PKG. Based on trust delegation between users in our system, KC-ABE offers users the option to decrypt

ciphertexts using either the private key from the PKG or user private keys from trusted users. This scheme enhances flexibility for users in obtaining decryption keys, thereby reducing the workload burden on the PKG.

In KC-ABE, we extend the capabilities of trusted users, enabling them to generate and distribute user private keys to other users. These cooperative user private keys possess equivalent decryption compared with private keys from the PKG when satisfying the cooperative access policy set by the encryptor. We propose a detailed KC-ABE scheme based on [23] and provide a formal proof of security for our construction within a well-defined security model.

KC-ABE stands out as a distinct and innovative solution, differing from the aforementioned approaches by its practicality, lightweight and extensibility. Specifically, the main contributions of our work are summarized as follows:

- *Practicality.* Our solution avoids the need for additional entities and eliminates the associated overheads of storage, communication, and updates. This innovative approach provides key management for dynamic, large-scale user systems, offering enhanced practicality.
- *Lightweight.* In our KC-ABE, we transfer the majority of key generation costs from the PKG to trusted users. As a result, the PKG concentrates on generating private keys for only a small number of users, leading to a substantial improvement in key management.
- *Extensibility.* Our system expands key delegation from a one-to-one model to key cooperation in a many-to-one framework. Leveraging the extended trust among users, the majority of users can obtain decryption keys from cooperative trusted users.

1.2 Related Work

In 1984, Shamir [26] introduced identity-based cryptography, with practical identity-based encryption (IBE) schemes emerging through the pioneering work of Boneh and Franklin [3]. Subsequently, Sahai and Waters proposed fuzzy IBE and ABE in [24]. Since then, numerous variants of ABE have been developed, such as revocable ABE [13, 6], accountable ABE [17, 19], policy-hiding ABE [15, 20], multi-authority ABE [5, 18, 16], and HABE [27].

HABE can be adopted to realize hierarchical access control by introducing hierarchical attribute authorities. It allows higher-level attribute authorities to delegate private keys to their subordinates and conduct private key transmission locally. Wang et al. [27] proposed hierarchical ABE schemes by combing ABE and hierarchical IBE introduced by Horwitz and Lynn [12].

Chase proposed the first multi-authority scheme in [5]. Lin et al. [18] focused on removing the central authority from the multi-authority ABE scheme. Then Lewko and Waters [16] proposed a new multi-authority ABE system. Any party can become an authority and there is no requirement for any global coordination.

In 2018, Garg et al. [8] introduced registration-based encryption(RBE) to address the key escrow problem in IBE. In 2023, Hohenberger et al. [10] proposed

registered ABE, which removed the need for long-term secret keys in the context of ABE. Moreover, the KC is entirely transparent and maintains no secrets but with inefficient key generations and key updating costs.

In 2014, Hohenberger and Waters proposed online/offline ABE [11] to address the huge encryption and key generation costs scale with the complexity of the access policy or the number of attributes. They developed new techniques for ABE that split the computation into offline and online phases.

When given a private key for attribute set A, a user can generate a new private key for any subset $B \subseteq A$, known as key delegation abuse in ABE schemes [2, 9, 28]. Additionally, Jiang et al. [14] proposed an ABE scheme against the issue of key delegation abuse. Unlike the one-to-one model of key delegation abuse, our KC-ABE extends to key cooperation from many-to-one. Specifically, these multiple low-level delegated keys process equivalent decryption with a high-level private key when satisfying the cooperative access policy set by the encryptor.

2 Definitions

The web of trust model [22, 1] is formed by expanding trust delegations illustrated between users in Fig.1. For example, Alice places direct trust in her friend Bob, who in turn extends direct trust to stranger Paul. Therefore, Alice can indirectly trust Paul. This allows users to communicate with known parties rather than rely exclusively on a certificate authority. Inspired by its trust delegation and decentralized nature, we consider a novel system in which users can obtain decryption keys from trusted users to distribute PKG authority and reduce its workload.

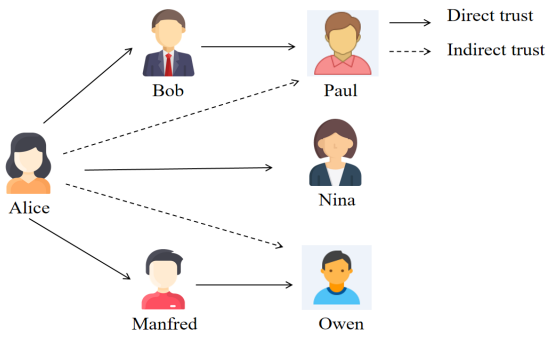


Fig. 1. A web of trust model

As shown in Fig.2, users with attribute set A_i and identity ID_i who obtain the private key $sk_{A_i}^{ID_i}$ directly from the PKG are called high-level users. However, in cases where the PKG becomes overwhelmed and is unable to respond to an increasing number of private key requests in time, a user with an attribute set

B encounters difficulties in establishing a direct connection with the PKG. In such instances, this user (referred to as a low-level user) has the alternative of obtaining user private keys $dk_B^{ID_i}$ from trusted high-level users based on his/her direct trust of the web of trust model. The specific process unfolds as follows in the right half of Fig.2.

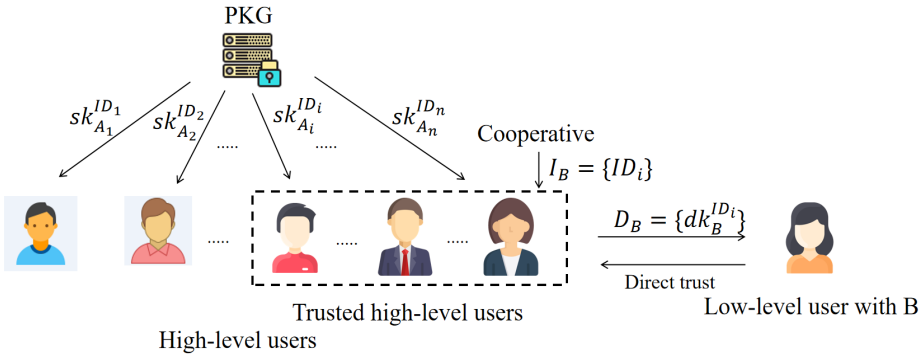


Fig. 2. Key Cooperative ABE system model

A low-level user with an attribute set B first determines a cooperative group (dotted box) identity set $I_B = \{ID_i\}$ of his/her trusted high-level users based on direct trust. Each trusted high-level user in I_B, with the private key $sk_{A_i}^{ID_i}$ from the PKG, generates a user private key $dk_B^{ID_i}$ under $B \subseteq A_i$. These cooperative trusted high-level users produce $|I_B|$ user private keys, consisting a set $D_B = \{dk_B^{ID_i}\}$ to the low-level user.

Therefore there are two types of decryption keys in our system: the private key $sk_{A_i}^{ID_i}$ generated from the PKG to a high-level user; user private keys $\{dk_B^{ID_i}\}$ issued from trusted high-level users to a low-level user. To enable user private keys for decryption, we introduce a valid cooperation minimum l within the ciphertext. Specifically, successful key cooperation for a low-level user is achieved when the number of user private keys from trusted high-level users, denoted as $|I_B|$, is not less than l. This means that the cooperation set of user private keys $D_B = \{dk_B^{ID_i}\}$ possess commensurate decryption with the private key $sk_{A_i}^{D_i}$.

It is essential to emphasize that the users' identities, represented by ID_i, are only associated with their private keys and they do not directly participate in the decryption process itself. Instead, these identities play a role between trusted high-level users during the generation of cooperative keys.

2.1 Algorithm Definitions

Different from basic ABE, decryption keys in KC-ABE consist of private keys from the PKG (PKeyGen) and user private keys from trusted high-level users

(UKeyGen). Correspondingly, there are two decryption algorithms PDec and UDec. Moreover, the ciphertext with an attribute set $A^{'}$ in our KC-ABE is embedded in the key cooperation minimum l. And the ciphertext can be decrypted using either a private key under $A^{'} \subseteq A_i$ or user private keys under $A^{'} \subseteq B$ and $l \leq |I_B|$. Hence, based on the basic ABE algorithm definitions, we define our key cooperative ABE consisting of six algorithms: Setup, PKeyGen and PDec, Enc, UKeyGen and UDec.

The algorithms in our KC-ABE are defined as follows:

- **Setup**$(\lambda, U, n) \rightarrow (mpk, msk)$. The Setup algorithm is executed by the PKG and takes as input the security parameter λ, the attribute universal set U and the number of users n. It outputs the master public key and master secret key (mpk, msk).

- **PKeyGen**$(mpk, msk, ID_i, A_i) \rightarrow sk_{A_i}^{ID_i}$. The PKeyGen algorithm is executed by the PKG and takes as input the master public key, master secret key mpk, msk, the user identity ID_i and attribute set A_i. It returns the private key $sk_{A_i}^{ID_i}$.

- **UKeyGen**$(mpk, ID_i, sk_{A_i}^{ID_i}, I_B = \{ID_i\}, B \subseteq A_i) \rightarrow dk_B^{ID_i}$. The UKeyGen algorithm is executed by a trusted high-level user with identity ID_i and takes as input identity ID_i, private key $sk_{A_i}^{ID_i}$, the trusted high-level user identity set $I_B, |I_B| < n$. It also inputs the master public key mpk, attribute set $B \subseteq A_i$, and returns the user private key $dk_B^{ID_i}$.

- **Enc**$(M, mpk, A^{'}, l) \rightarrow CT$. The Enc algorithm is executed by the encryptor and takes as input a message M, the master public key mpk, the attribute set $A^{'}$, and the key cooperative minimum $l < n$. It returns the ciphertext CT.

- **PDec**$(sk_{A_i}^{ID_i}, CT) \rightarrow M$. The PDec algorithm is executed by a high-level user and takes as input the private key $sk_{A_i}^{ID_i}$ and the ciphertext CT. It outputs the message M if $A^{'} \subseteq A_i$.

- **UDec**$(D_B = \{dk_B^{ID_i}\}, CT) \rightarrow M$. The UDec algorithm is executed by a low-level user and takes as input a set D_B of user private keys $\{dk_B^{ID_i}\}$ and the ciphertext CT. It outputs the message M if $A^{'} \subseteq B$ and $l \leq \mid I_B \mid$.

Correctness: We require the correctness of the KC-ABE are as follows:

- **PDec Correctness:** For any $(mpk, msk) \leftarrow \text{Setup}(\lambda, U, n)$, ciphertext $CT \leftarrow \text{Enc}(M, mpk, A^{'}, l)$ and private key $sk_{A_i}^{ID_i} \leftarrow \text{PKeyGen}(mpk, msk, ID_i, A_i)$, if $A^{'} \subseteq A$, the PDec algorithm always outputs the correctness message M. Otherwise, the message in CT cannot be decrypted using $sk_{A_i}^{ID_i}$.

- **UDec Correctness:** For any $(mpk, msk) \leftarrow \text{Setup}(\lambda, U, n)$, ciphertext $CT \leftarrow \text{Enc}(M, mpk, A^{'}, l)$ and $|I_B|$ user private keys $D_B = \{dk_B^{ID_i}\}$, $dk_B^{ID_i} \leftarrow \text{UKeyGen}(mpk, ID_i, sk_{A_i}^{ID_i}, I_B = \{ID_i\}, B \subseteq A_i)$, if $A^{'} \subseteq B$ and $l \leq |I_B|$, that means if $A^{'} \subseteq B$ and the number of trusted high-level users in the group I_B is not less than the key cooperative minimum l decided by the encryptor, the UDec algorithm always outputs the correctness message M. Otherwise, the message in CT cannot be decrypted using $D_B = \{dk_B^{ID_i}\}$.

Discussion. Based on the KC-ABE system model in Fig. 2 and algorithm definitions provided above, we now present a comparison between our KC-ABE proposal and other relevant ABE schemes in three aspects: whether additional entities are introduced, who generates the decryption key and whether there is key delegation. We discuss them as follows.

Table 1. Comparison with related ABE schemes.

Schemes	Additional entities	Key generation	Key delegation
HABE [27]	Domain AAs	Domain AAs	–
Multi-authority ABE [5]	Multiple AAs	Multiple AAs	–
Registered ABE [10]	KC	KC & Users	–
Online/Offline ABE [11]	–	PKG	–
Key delegation ABE [2, 9, 28]	–	PKG	Single key delegation
KC-ABE	–	PKG or Trusted users	Trust user delegation

As shown in Table 1, one straightforward approach to address the huge workload on the PKG is to introduce additional entities aimed at decentralizing the PKG's power in [27, 5, 10]. However, introducing other entities still requires users to depend heavily on them to generate decryption keys. In contrast, our approach differs in that we directly enhance the capabilities of existing trusted users and avoid the need for additional entities. It eliminates the associated overheads of storage, communication, and updates, which holds more practical significance.

Another method is splitting key generations into two phases in online/offline ABE [11]. However, the PKG in online/offline ABE still generates final private keys for all users. Different from it, we transfer most of the key generation costs completely from the PKG to trusted users. Consequently, the PKG's role now primarily focuses on generating private keys for a small number of users, resulting in lightweight key management of PKG.

Early classic CP-ABE schemes [2, 9, 28] allowed one-to-one key delegation based on the subset relation of users' attribute sets. Our KC-ABE expands the scope of key delegation beyond its limitations. Based on trust relationships between users, multiple trusted users can generate decryption keys for a user, facilitating extensible key delegation throughout the system.

In conclusion, our KC-ABE represents a unique solution that distinguishes itself from existing approaches by its practicality, lightweight, and extensibility. This innovative approach offers a novel solution to the huge key management of the PKG in ABE, with a summary of the results presented in Table 1.

2.2 Security Model

The security model is built from respective of semantic security against the *Indistinguishability under selective chosen plaintext attacks* (IND-sCPA). More precisely, KC-ABE security is defined by the following game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. $\mathcal{A}$ specifies the target attribute set A^* and the key cooperative minimum l^* in advance. The game says that it is indistinguishable from deciding the message in a ciphertext for any attribute set A^* and any private key $sk_{A_i}^{ID_i}$ if $A^* \not\subseteq A_i$ holds. Additionally, it also says that it is indistinguishable to decide the message in a ciphertext for the attribute set A^*, the key cooperative minimum l^* and any $|I_B|$ user private keys $D_B = \{dk_B^{ID_i}\}$ if $l^* > |I_B|$ holds under $A^* \subseteq B$.

- **Initialization.** $\mathcal{C}$ chooses an attribute universal set U and the numer of users n. $\mathcal{A}$ declares the challenge attribute set $A^* \subseteq U$ and the challenge key cooperative minimum $l^* < n$, which it will try to attack, and sends them to $\mathcal{C}$.

- **Setup.** $\mathcal{C}$ runs the Setup algorithm to generate the master public key and master secret key (mpk, msk) with a security parameter λ. It sends mpk to the adversary $\mathcal{A}$. $\mathcal{C}$ keeps msk to respond to private key queries and user private key queries.

- **Query phase 1.**
  1. $\mathcal{A}$ makes q_1 private key queries with attribute sets $A_i \subseteq U$ and user identities ID_i, $i \in \{1, \ldots, q_1\}$.
  2. $\mathcal{A}$ makes q_2 user private key queries with attribute sets $B_j \subseteq U$ and trusted high-level user identity sets I_{B_j}, $j \in \{1, \ldots, q_2\}$.

- **Challenge.** $\mathcal{A}$ outputs two equal length messages M_0, M_1. It gives the attribute set A^* and the key cooperative minimum l^* with following restrictions:
  1. $A^* \not\subseteq A_1, \ldots, A_{q_1}$.
  2. $l^* > |I_{B_j}|, I_{B_j} = \{ID_i\}, j = 1, \ldots, q_2$ under $A^* \subseteq B_1, \ldots, B_{q_2}$.
  $\mathcal{C}$ flips a random coin b, and encrypts M_b. The ciphertext CT^* is given to the adversary $\mathcal{A}$.

- **Query phase 2.** Phase 2 is repeated with the same restrictions as follows:
  1. $\mathcal{A}$ makes $q_1^{'}$ private key queries that attribute sets $A_i \subseteq U$ satisfy $A^* \not\subseteq A_i$ and user identities ID_i, $i \in \{1, \ldots, q_1^{'}\}$.
  2. $\mathcal{A}$ makes $q_2^{'}$ user private key queries that I_{B_j} satisfy $l^* > |I_{B_j}|$ under $A^* \subseteq B_j$, $j \in \{1, \ldots, q_2^{'}\}$.

- **Guess.** $\mathcal{A}$ outputs a guess $b^{'}$ of b and wins the game if $b^{'} = b$.

The advantage of an adversary $\mathcal{A}$ in the IND-sCPA game is defined as follows:

$$\mathsf{Adv}_{\mathsf{KC\text{-}ABE}}^{\mathsf{IND\text{-}sCPA}}(\mathcal{A}) = |\Pr[b' = b] - \frac{1}{2}|$$

Definition 1. *We say the KC-ABE scheme is selectively secure against chosen plaintext attacks if all polynomial-time adversaries $\mathcal{A}$ have at most a negligible advantage in the above security game.*

3 Constructions

In this section, we present our realization of key cooperative scheme of CP-ABE. We begin with an overview of our KC-ABE scheme. Subsequently, we review the first efficient and secure two-party identity-based non-interactive key exchange (IB-NIKE) scheme [21], the Sakai-Ohgishi-Kasahara (SOK) scheme [25]. Lastly, we show the construction of a detailed CP-ABE scheme for our key cooperative ABE, based on the practical CP-ABE scheme proposed in [23] and the SOK scheme.

3.1 Overview

Building on the practical CP-ABE scheme presented in [23], we propose a key cooperative CP-ABE scheme but simplify the access policy of the scheme in [23] from the Linear Secret Sharing Scheme (LSSS) to a subset structure, i.e., $A^{'} \subseteq A_i$ or $A^{'} \subseteq B$. Additionally, in our construction, the encryptor possesses an attribute set of $A^{'}$ and specifies the minimum of user private key cooperation, denoted as l.

To achieve cooperation of user private keys within I_B, each trusted high-level user with A_i and ID_i first generates a cooperative key ck_i^B. We should guarantee that only the combination of $|I_B|$ cooperative keys is a specific and deterministic value for I_B, which we set to $g^{|I_B|}$. Furthermore, $g^{|I_B|}$ can be eliminated when the attribute set B satisfies the access structure ascribed in the ciphertext. It is worth noting that the low-level user will send the identity set I_B to the PKG in the SOK scheme. The PKG will verify the trusted high-level user with identity $ID_i \in I_B$ during the generation of shared keys and prevent unauthorized users from forging any shared key.

Our system prevents collusion attacks by tying a user's attributes together using different random values for different users. This makes it impossible to successfully combine these attributes with another user's attributes during decryption. Collusion attacks won't help since attribute mappings are randomized to the randomness of a particular user's private key. Furthermore, the user private keys of trusted high-level users are re-randomized, making it impossible to combine different user private keys belonging to different users.

3.2 Sakai-Ohgishi-Kasahara Scheme

We first review the IB-NIKE [21] scheme: SOK scheme [25]. IB-NIKE is an analog of non-interactive key exchange (NIKE) [7] in the identity-based setting, which enables a group of users registered in the same PKG to agree on a unique shared key without any interaction. The SOK scheme is as follows:

- **Setup**$(1^k) \rightarrow (params, msk)$. The Setup takes as input security parameter 1^k and obtains $(\mathbb{G}, \mathbb{G}_T, e)$. The PKG picks a random $\beta \in \mathbb{Z}_p$ as master secret key msk and two hash functions $H_1 : \{0,1\}^* \rightarrow \mathbb{G}$, $H_2 : \mathbb{G}_T \rightarrow \mathbb{Z}_p$. It also outputs system parameters $params = (\mathbb{G}, \mathbb{G}_T, e, g, H_1, H_2)$.

- **Extract**$(params, msk, ID \in \{0,1\}^*) \rightarrow S_{ID}$. The Extract algorithm is run from the PKG with ID and inputs $params$ and msk. The PKG first verifies the validity of identity ID and then outputs the private key $S_{ID} = H_1(ID)^{\beta}$.

- **SharedKey**$(params, S_{ID_A}, ID_B \in \{0,1\}^*) \rightarrow K_{A,B}$. The SharedKey generation algorithm takes as input identity $ID_B \in \{0,1\}^*$ and $params$. It inputs private key $S_{ID_A} = H_1(ID_A)^{\beta}$ and returns the shared key $K_{A,B} = H_2(e(H_1(ID_A)^{\beta}, H_1(ID_B)))$.

Correctness: We require that, for any pair of identities ID_A, ID_B, and corresponding private keys S_{ID_A}, S_{ID_B}, **SharedKey** satisfies the correctness:

$$\textbf{SharedKey}(params, S_{ID_A}, ID_B) = \textbf{SharedKey}(params, S_{ID_B}, ID_A).$$

This ensures that users A and B can indeed generate a shared key $K_{A,B}$ without any interaction.

3.3 Scheme

In this section, we will show the KC-ABE scheme based on the CP-ABE presented in [23] and the SOK scheme. The KC-ABE scheme comprises six algorithms. Specifically, the Setup and PKeyGen algorithms are executed by the PKG. The UKeyGen algorithm is employed by each trusted high-level user to generate a user private key $dk_B^{ID_i}$ for a low-level user, utilizing the private key $sk_{A_i}^{ID_i}$ and the cooperative key associated with identity ID_i, following the execution of the SOK scheme detailed in Section 3.2. The encryptor encrypts the message and the attribute set during the Enc algorithm. Users have the option to choose different decryption algorithms based on the type of decryption keys, either PDec or UDec.

Note that users' identities ID_i are only associated with their private keys and are used to generate cooperative keys between trusted high-level users. However, they do not directly partake in the decryption process. The KC-ABE scheme is structured as follows:

- **Setup**$(\lambda, U, n) \rightarrow (mpk, msk)$. The Setup algorithm takes as input the security parameter λ, the universal attribute set U and the number of users n. It randomly chooses $g, w, v, \{h_k\} \in \mathbb{G}, k \in \{1, \ldots, n\}, \{u_j\} \in \mathbb{G}, j \in \{1, \ldots, |U|\}$ and $\alpha \in \mathbb{Z}_p$. The Setup algorithm outputs the master public key and master secret key (mpk, msk) as follows.

$$mpk = (g, w, v, \{h_k\}_{k \in \{1,\ldots,n\}}, \{u_j\}_{j \in \{1,\ldots,|U|\}}, e(g,g)^{\alpha}), msk = \alpha.$$

- **PKeyGen**$(mpk, msk, ID_i, A_i) \rightarrow sk_{A_i}^{ID_i}$. The PKeyGen algorithm takes as input attribute set A_i, user identity ID_i and master public key, master secret key mpk, msk. It randomly chooses $r_i \in \mathbb{Z}_p$. The PKeyGen algorithm returns the private key $sk_{A_i}^{ID_i}$ as follows.

$$sk_{A_i}^{ID_i} = (g^{\alpha} w^{r_i}, g^{r_i}, \{u_j^{-r_i}\}_{j\in\{1,\ldots,|A_i|\}}, \{h_k^{r_i}\}_{k\in\{1,\ldots,n\}}).$$

- **UKeyGen**$(mpk, ID_i, sk_{A_i}^{ID_i}, I_B = \{ID_i\}, B \subseteq A_i) \rightarrow dk_B^{ID_i}$. The UKeyGen algorithm takes as input the master public key mpk, the private key $sk_{A_i}^{ID_i}$ of trusted high-level user with identity ID_i, the low-level user's attribute set $B \subseteq A_i$, the trusted high-level user identity set $I_B = \{ID_i\}$ and $|I_B| < n$. The trusted high-level user with ID_i first generates a cooperative key ck_i^B, through the following process:
  1. Request private key S_{ID_i} for the PKG that inputs the system parameters $params$ and master secret key msk in the SOK scheme and then obtain $S_{ID_i} = H_1(ID_i)^{\beta}$.
  2. Execute the SharedKey algorithm in the SOK scheme by inputting $params$, private key S_{ID_i}, identity $ID_j \in I_B, j \neq i$. The trusted high-level user with ID_i returns shared key $K_{i,j} = H_2(e(H_1(ID_i)^{\beta}, H_1(ID_j)))$.
  3. Repeat the SharedKey algorithm $|I_B|-1$ times and then obtain $|I_B|-1$ shared keys $K_{i,j}, j \neq i, j \in \{1, \ldots, |I_B|\}$ between the trusted high-level user with ID_i and every user else in the cooperative group I_B.
  4. For $j \in \{1, \ldots, |I_B|\}$ and $j \neq i$, the trusted high-level user with ID_i sets $a_{i,j}$ as follows:

$$a_{i,j} = \begin{cases} -K_{i,j} & i > j \\ K_{i,j} & i < j \end{cases}$$

  Trusted high-level user with ID_i compacts these $|I_B| - 1$ shared keys into a cooperative key $ck_i^B = g^{x_i} = g^{1+\sum_{j=1, j\neq i}^{|I_B|} a_{i,j}}$.

  Then the trusted high-level user with identity ID_i randomly chooses $y_i \in \mathbb{Z}_p$ and $r_i^{'} = r_i + y_i$. The UKeyGen algorithm returns the user private key $dk_B^{ID_i}$ as follows.

$$dk_B^{ID_i} = (g^{\alpha} w^{r_i} \cdot g^{x_i} \cdot (\prod_{k=1}^{n-|I_B|} h_k)^{r_i} \cdot (\prod_{k=1}^{n-|I_B|} h_k w)^{y_i}, g^{r_i} \cdot g^{y_i},$$
$$\{u_j^{-r_i} \cdot u_j^{-y_i}\}_{j\in\{1,\ldots,|B|\}}, \{h_k^{r_i} \cdot h_k^{y_i}\}_{k\in\{n-|I_B|+1,\ldots,n\}}).$$

- **Enc**$(M, mpk, A^{'}, l) \rightarrow CT$. The Enc algorithm takes as input a message M, the master public key mpk, the attribute set $A^{'}$, and the key cooperative minimum $l < n$. It randomly chooses $s, z, z_j, t_j \in \mathbb{Z}_p$ and $\sum_{j=1}^{|A^{'}|} z_j = z$. The Enc algorithm returns the ciphertext CT as follows.

$$CT = (Me(g,g)^{\alpha s}, g^s, w^s v^z, (\prod_{k=1}^{n-l} h_k)^s, \{u_j^{t_j} v^{z_j}, g^{t_j}\}_{j\in\{1,\ldots,|A^{'}|\}}).$$

- **PDec**$(sk_{A_i}^{ID_i}, CT) \rightarrow M$. The PDec algorithm takes as input the private key $sk_{A_i}^{ID_i}$ and the ciphertext CT. It outputs the message if $A^{'} \subseteq A_i$. If $A^{'} \subseteq A_i$, we can computer as follows:

$$\frac{Me(g,g)^{\alpha s}e(w^s v^z, g^{r_i})}{e(g^{\alpha}w^{r_i}, g^s)\prod_{j=1}^{|A^{'}|} e(u_j^{t_j}v^{z_j}, g^{r_i})e(u_j^{-r_i}, g^{t_j})} = M.$$

- **UDec**$(D_B = \{dk_B^{ID_i}\}, CT) \rightarrow M$. The UDec algorithm takes as input the $|I_B|$ user private keys $D_B = \{dk_B^{ID_i}\}$ and the ciphertext CT. It outputs the message if $A^{'} \subseteq B$ and $l \leq |I_B|$.

  First, if $A^{'} \subseteq B$, same as in PDec algorithm, we obatin $e(v,g)^{zr_i^{'}}$. Then if $l \leq |I_B|$, we can computer $e(g,g)^{\alpha s}$ as follows:

$$\left(\prod_{i=1}^{|I_B|} \frac{e(g^{\alpha+x_i}(\prod_{k=1}^{n-|I_B|} h_k w)^{r_i^{'}}, g^s)\prod_{k=n-|I_B|+1}^{n-l} e(h_k^{r_i^{'}}, g^s)e(v,g)^{zr_i^{'}}}{e(\prod_{k=1}^{n-l} h_k^s \cdot w^s v^z, g^{r_i^{'}})e(g^{|I_B|}, g^s)}\right)^{\frac{1}{|I_B|}}.$$

  At last,

$$\frac{Me(g,g)^{\alpha s}}{e(g,g)^{\alpha s}} = M.$$

Correctness:

- **PDec Correctness:** For attribute set A_i in the private key $sk_{A_i}^{ID_i}$, satisfying $A^{'} \subseteq A_i$, we can compute

$$\prod_{j=1}^{|A^{'}|} e(u_j^{t_j}v^{z_j}, g^{r_i})e(u_j^{-r_i}, g^{t_j}) = \prod_{j=1}^{|A^{'}|} e(v^{z_j}, g^{r_i}) = e(v^z, g^{r_i}).$$

  Thus $e(g,g)^{\alpha s}$ can be computed from

$$\frac{e(g^{\alpha}w^{r_i}, g^s)e(v^z, g^{r_i})}{e(w^s v^z, g^{r_i})} = e(g,g)^{\alpha s}.$$

- **UDec Correctness:** For attribute set B in $|I_B|$ user private keys $D_B = \{dk_B^{ID_i}\}$, satisfying $A^{'} \subseteq B$, for each user private key $dk_B^{ID_i}$, we can compute

$$\prod_{j=1}^{|A^{'}|} e(u_j^{t_j}v^{z_j}, g^{r_i^{'}})e(u_j^{-r_i^{'}}, g^{t_j}) = \prod_{j=1}^{|A^{'}|} e(v^{z_j}, g^{r_i^{'}}) = e(v^z, g^{r_i^{'}}).$$

  Then $e(g,g)^{\alpha s}$ will be computed under the number of trusted high-level users $|I_B| \geq l$ as following:

$$\prod_{i=1}^{|I_B|} \frac{e(g^{\alpha+x_i}(\prod_{k=1}^{n-|I_B|} h_k w)^{r_i^{'}}, g^s)\prod_{k=n-|I_B|+1}^{n-l} e(h_k^{r_i^{'}}, g^s)e(v,g)^{zr_i^{'}}}{e(\prod_{k=1}^{n-l} h_k^s \cdot w^s v^z, g^{r_i^{'}})}$$

$$= \prod_{i=1}^{|I_B|} \frac{e(g^{\alpha+x_i}, g^s)e(w^{r_i^{'}}, g^s)e((\prod_{k=1}^{n-|I_B|} h_k)^{r_i^{'}}, g^s) \prod_{k=n-|I_B|+1}^{n-l} e(h_k^{r_i^{'}}, g^s)e(v, g)^{zr_i^{'}}}{e(\prod_{k=1}^{n-l} h_k^s \cdot w^s v^z, g^{r_i^{'}})}$$

$$= \prod_{i=1}^{|I_B|} \frac{e(g^{\alpha+x_i}, g^s)e(w^{r_i^{'}}, g^s) \prod_{k=1}^{n-l} e(h_k^{r_i^{'}}, g^s)e(v, g)^{zr_i^{'}}}{e(\prod_{k=1}^{n-l} h_k^s, g^{r_i^{'}})e(w^s, g^{r_i^{'}})e(v^z, g^{r_i^{'}})} = \prod_{i=1}^{|I_B|} e(g^{\alpha+x_i}, g^s).$$

Here we show how to achieve key cooperation in the KC-ABE scheme that the multiplication of $|I_B|$ cooperative keys is $\prod_{i=1}^{|I_B|} ck_i^B = g^{|I_B|}$.
From the generation of cooperative key ck_i^B in UKeyGen algorithm, trusted high-level user with ID_i sets $ck_i^B = g^{x_i} = g^{1+\sum_{j=1,j\neq i}^{|I_B|} a_{i,j}}$. It is important to note the following points:

1. In the key cooperative group I_B, for $i, j \in \{1, \ldots, |I_B|\}, j \neq i$, when $i > j$, $a_{i,j}$ in ck_i^B is $-K_{i,j}$ and $a_{j,i}$ in ck_j^B is $K_{j,i}$; conversely when $i < j$.
2. Because of shared keys in the SOK scheme satisfying $K_{i,j} = K_{j,i}$, thus $-K_{i,j}, K_{j,i}$ can be eliminated to zero when combining cooperative keys ck_i^B, ck_j^B.

Therefore all $a_{i,j}, a_{j,i}$ can be eliminated to zero and achieve $g^{|I_B|}$ when combining $|I_B|$ cooperative keys $ck_i^B, i \in \{1, \ldots, |I_B|\}$. Thus,

$$\prod_{i=1}^{|I_B|} e(g^{\alpha+x_i}, g^s) = \prod_{i=1}^{|I_B|} e(g^{\alpha+1+\sum_{j=1,j\neq i}^{|I_B|} a_{i,j}}, g^s)$$

$$= \prod_{i=1}^{|I_B|} e(g^{\alpha} \cdot g^{1+\sum_{j=1,i>j}^{|I_B|} -K_{i,j}+\sum_{j=1,i<j}^{|I_B|} K_{i,j}}, g^s) = e(g, g)^{|I_B|s(\alpha+1)}$$

Thus $e(g, g)^{\alpha s}$ can be computed from

$$(\frac{(e(g, g)^{|I_B|s(\alpha+1)})}{e(g^{|I_B|}, g^s)})^{\frac{1}{|I_B|}} = e(g, g)^{\alpha s}.$$

Example. Here, we provide a simple example of key cooperation in our KC-ABE scheme. Suppose the low-level user with attribute set B determines an identity set of trusted high-level users $I_B = \{ID_1, ID_2, ID_3\}$. The trusted high-level user with ID_1 generates the shared key $ck_1^B = g^{1+K_{1,2}+K_{1,3}}$. Similarly, trusted high-level users with ID_2, ID_3 also generate $ck_2^B = g^{1-K_{2,1}+K_{2,3}}, ck_3^B = g^{1-K_{3,2}-K_{3,1}}$, respectively. Because of $K_{i,j} = K_{j,i}, i, j \in \{1, 2, 3\}, i \neq j$, it it easy to see that $\prod_{i=1}^{3} ck_i^B = g^{|I_B|} = g^3$.

4 Security Analysis

4.1 Complexity Assumption

Decisional $(q-1)$ **Assumption [23].** For our construction, we will use a $q-1$ type assumption in [23], which is similar to the Decisional Parallel Bilinear Diffie-Hellman Exponent Assumption [28]. This assumption can be proved secure in the generic group model and is defined via the following game between $\mathcal{C}$ and $\mathcal{A}$:

Initially $\mathcal{C}$ calls the group generation algorithm with input the security parameter, picks a random group element $g \in \mathbb{G}$, and $q+2$ random exponents $a, s, \overrightarrow{b} = \{b_1, \ldots, b_q\} \in \mathbb{Z}_p$. Then it sends to $\mathcal{A}$ the group description $(p, \mathbb{G}, \mathbb{G}_p, e)$ and all of the following terms:

$$
\begin{aligned}
&g, g^{s}\\
&g^{a^{i}}, g^{b_j}, g^{sb_j}, g^{a^{i}b_j}, g^{\frac{a^{i}}{(b_j)^2}}, \forall (i,j) \in [q,q]\\
&g^{\frac{a^{i}b_j}{(b_{j'})^2}}, \forall (i,j,j') \in [2q,q,q], j \neq j^{'}\\
&g^{\frac{a^{i}}{b_j}}, \forall (i,j) \in [2q,q], i \neq q+1\\
&g^{\frac{sa^{i}b_j}{b_{j'}}}, g^{\frac{sa^{i}b_j}{(b_{j'})^2}} \forall (i,j,j^{'}) \in [q,q,q], j \neq j^{'}.
\end{aligned}
$$

$\mathcal{C}$ flips a random coin $b \in \{0,1\}$ and if $b=0$, it gives to $\mathcal{A}$ the term $T = e(g,g)^{sa^{q+1}}$. Otherwise, it gives a random term $T = R \in \mathbb{G}_T$. Finally the $\mathcal{A}$ outputs a guess $b^{'} \in \{0,1\}$.

4.2 Security Proof

We used the SOK scheme in user private key generation of trusted high-level users. Paterson [21] established a new security model for IB-NIKE and then proved the security of the SOK scheme in the extended security model. As mentioned before in Section 3.3 scheme, $a_{i,j}$ actually are shared keys between users with ID_i and ID_j in the SOK scheme. The final cooperative key ck_i^B in our KC-ABE scheme for the user with ID_i is just the combination of different $a_{i,j}$. This setting would not influence the security level when using the SOK scheme. Due to the original SOK scheme's indistinguishability of shared key (IND-SK) secure under Bilinear Diffie-Hellman(BDH) assumption in the extended security model in [21], thus our generation of cooperative keys is secure. Therefore, we only prove the security of the KC-ABE scheme in Section 3.3 as follows.

Theorem 1. *If the* $q-1$ *assumption in [23] holds then no polynomial adversary can selectively break our scheme with a challenge attribute set* A^* *and a challenge key cooperative minimum* l^*.

Proof. Suppose there exists an adversary $\mathcal{A}$ who breaks the proposed KC-ABE scheme with $\mathsf{Adv}^{\mathsf{IND\text{-}sCPA}}_{\mathsf{KC\text{-}ABE}}(\mathcal{A}) = \epsilon$. We built a simulator $\mathcal{B}$ that has advantage $\frac{\epsilon}{2}$ in solving the $q-1$ assumption in [23]. Given as input an assumption instance, $\mathcal{B}$ plays the role of the challenger $\mathcal{C}$ in the game, and interacts with the adversary $\mathcal{A}$ as follows.

– **Initialization.** $\mathcal{B}$ first chooses the attribute universal set U and the number of users n. The selective game with $\mathcal{A}$ outputs a challenge attribute set A^* and a challenge key cooperative minimum $l^*, l^* < n \ll q$ that it intends to attack.

– **Setup.** To generate the system parameters, $\mathcal{B}$ picks randoms $\tilde{\alpha}$ and sets $\alpha = a^{q+1} + \tilde{\alpha}$. Notice that this way α is correctly distributed and information-theoretically hidden from $\mathcal{A}$.
$\mathcal{B}$ sets $w = g^a, v = g^{\frac{-a}{b_q}} \cdot g^{\frac{a}{b_1}}$ and picks randoms $w_1, \ldots, w_{|U|}$ in $\mathbb{Z}_p$ and $\gamma_1, \ldots, \gamma_n$ in $\mathbb{Z}_p$. Then $\mathcal{B}$ sets $\{u_j\}, \{h_k\}$ as follows.

$$u_j = g^{\frac{a^{q-j+1} - a^2 w_j}{b_q}}, j = 1, \ldots, |U|. \quad h_k = g^{\frac{\gamma_k a^{k+1} b_{n-l^*+1}}{(b_k)^2}}, k = 1, \ldots, n.$$

Finally, $\mathcal{B}$ gives $\mathcal{A}$ $mpk = (g, w, v, h_1, \ldots, h_n, u_1, \ldots, u_{|U|})$. Observe that all these values are distributed uniformly and independently in $\mathbb{G}$ as required. The master key corresponding to the system paramaters is $g^\alpha = g^{a^{q+1}+\tilde{\alpha}}$, which is unknown to $\mathcal{B}$ since $\mathcal{B}$ does not have $g^{a^{q+1}}$.

– **Query phase 1.** The adversary $\mathcal{A}$ makes private key queries and user private key queries in this phase as follows.
1. The adversary $\mathcal{A}$ makes private key queries for non authorized sets A_i and user identity ID_i in this phase, $i \in \{1, \ldots, q_1\}$. For a private key query on A_i and ID_i. $\mathcal{B}$ picks randoms $\theta_j \in \mathbb{Z}_p$ for $a_j \in A^*$, $\sum_{j=1}^{|A^*|} \theta_j = 0$. $\mathcal{B}$ picks randoms $\tilde{r_i} \in \mathbb{Z}_p$ and sets

$$-r_i = \tilde{r_i} + a^q + \frac{(a^{q-1} + \sum_{i=1}^n w_i a^i)}{(\sum_{j=1}^{|S|} \theta_j)}$$

Notice that r_i is well-defined only for attribute sets in the specific unauthorized set A_i or unleaded attribute sets. At first, we set $S = A^* \bigcap A_i$ and only $\sum_{j=1}^{|A^*|} \theta_j = 0$. For the unauthorized set A_i, $A^* \not\subseteq A_i$, we obtain $S \neq A^*$ and $\sum_{j=1}^{|S|} \theta_j$ is not zero. Otherwise, for the authorized set A_i, $A^* \subseteq A_i$, we obtain $S = A^*$ and $\sum_{j=1}^{|S|} \theta_j$ is zero. Namely, the set of private keys is distributed in two types: the unauthorized, which the simulator can use the above method, and the authorized, which the simulator cannot create. This is properly distributed due to $\tilde{r_i}$. Then, using the suitable terms from the assumption, it calculates. The private key $sk_{A_i}^{ID_i}$ can be simulated as following:

$$g^\alpha w^{r_i} = g^{a^{q+1}+\tilde{\alpha}} \cdot g^{-a^{q+1}} \cdot g^{\frac{a^q + \sum_{i=1}^n w_i a^{i+1}}{-(\sum_{j=1}^{|S|} \theta_j)}} \cdot g^{-a\tilde{r_i}}, g^{r_i} = g^{-\tilde{r_i} - a^q + \frac{(a^{q-1} + \sum_{i=1}^n w_i a^i)}{-(\sum_{j=1}^{|S|} \theta_j)}},$$

$$\{h_k^{r_i}\} = \{h_k^{-\tilde{r_i}} \cdot g^{\frac{-\gamma_k a^{q+k+1} b_{n-l^*+1}}{(b_k)^2}} \cdot g^{\frac{\gamma_k (a^{q+k} + \sum_{t=1}^{n} \gamma_k w_t a^{t+k+1}) b_{n-l^*+1}}{-(\sum_{j=1}^{|S|} \theta_j)(b_k)^2}}\},$$

$$\begin{aligned}\{u_j^{-r_i}\} =& \{u_j^{-\tilde{r}_i} \cdot g^{\frac{a^{2q-j} + \sum_{i=1}^{n} w_i a^{q-j+i+1} - w_j a^{q+1} - \sum_{i=1}^{n} w_i w_j a^{i+2}}{b_q (\sum_{j=1}^{|S|} \theta_j)}} \cdot g^{\frac{a^{2q-j+1} - w_j a^{q+2}}{b_q}}\} \\ =& \{u_j^{-\tilde{r}_i} \cdot g^{\frac{a^{2q-j} + \sum_{i=1, i\neq j}^{n} w_i a^{q-j+i+1} - \sum_{i=1}^{n} w_i w_j a^{i+2}}{b_q (\sum_{j=1}^{|S|} \theta_j)}} \cdot g^{\frac{a^{2q-j+1} - w_j a^{q+2}}{b_q}}\},\end{aligned}$$

$\mathcal{B}$ sets private key $sk_{A_i}^{ID_i} = (g^{\alpha} w^{r_i}, g^{r_i}, \{h_k^{r_i}\}_{k\in\{1,\ldots,n\}}, \{u_j^{-r_i}\}_{j\in\{1,\ldots,|A_i|\}})$ as above and sends them to $\mathcal{A}$.

2. The adversary $\mathcal{A}$ also makes user private key queries in this phase for authorized attribute sets B_j, $A^* \subseteq B_j$ and unauthorized identity sets $I_{B_j} = \{ID_i\}, |I_{B_j}| < l^*$, $j \in \{1, \ldots, q_2\}$. For simplicity, we set B and I_B as attribute sets and identity sets in each user private key query, respectively. Here, for each user private key query on attribute set B and identity $ID_i \in I_B$, $\mathcal{B}$ first generates private keys $sk_{A_i'}^{ID_i}$ with satisfying attribute set A_i', $A^* \subseteq A_i'$ and identity $ID_i \in I_B$. $\mathcal{B}$ generates the user private key $dk_B^{ID_i}$ based on private keys $sk_{A_i'}^{ID_i}$ under $B \subseteq A_i'$. Here we set $\sum_{j=1}^{|S|} \theta_j = 0$ in this phase for each authorized attribute set B under $A^* \subseteq B$ and $S = A^* \cap B$. $\mathcal{B}$ picks a random $\tilde{y}_i \in \mathbb{Z}_p$ and sets

$$-y_i = \tilde{y}_i + a^{q-1}(\sum_{j=1}^{|S|} \theta_j + 1) + \sum_{i=1}^{n} w_i a^i - \frac{a^{q-(n-l^*)-1}}{\gamma_{n-l^*+1}}$$

$\mathcal{B}$ picks a random $\tilde{x_i}$ and sets a cooperative key $ck_i^B = g^{x_i} = g^{\tilde{x_i} - a^{q+1} - \frac{a^{q+1}}{b_{n-l^*+1}}}$. $\mathcal{B}$ sets

$$\Phi = (\prod_{k=1}^{n-|I_B|} h_k)^{-\tilde{y}_i - a^{q-1}(\sum_{j=1}^{|S|} \theta_j + 1) - \sum_{i=1}^{n} w_i a^i}$$

$\mathcal{B}$ computes $(\prod_{k=1}^{n-|I_B|} h_k)^{y_i}$

$$\begin{aligned}=& \Phi \cdot (g^{\frac{\gamma_1 a^2 b_{n-l^*+1}}{(b_1)^2} + \ldots + \frac{\gamma_{n-l^*+1} a^{n-l^*+2} b_{n-l^*+1}}{(b_{n-l^*+1})^2} + \ldots + \frac{\gamma_{n-|I_B|} a^{n-|I_B|+1} b_{n-l^*+1}}{(b_{n-|I_B|})^2}})^{\frac{a^{q-(n-l^*)-1}}{\gamma_{n-l^*+1}}} \\ =& \Phi \cdot (g^{\frac{\gamma_1 a^2 b_{n-l^*+1}}{(b_1)^2} + \ldots + \frac{\gamma_{n-|I_B|} a^{n-|I_B|+1} b_{n-l^*+1}}{(b_{n-|I_B|})^2}})^{\frac{a^{q-(n-l^*)-1}}{\gamma_{n-l^*+1}}} \cdot g^{\frac{a^{q+1}}{b_{n-l^*+1}}} \\ =& \Phi \cdot \Psi \cdot g^{\frac{a^{q+1}}{b_{n-l^*+1}}}.\end{aligned}$$

Notice that $(\prod_{k=1}^{n-|I_B|} h_k)^y$ is well-defined only for the unauthorized number of trusted high-level users $|I_B|$, $|I_B| < l^*$. Since if $|I_B| < l^*, n - |I_B| >$

$n - l^*$, then $\prod_{k=1}^{n-|I_B|} h_k$ includes $g^{\frac{\gamma_{n-l^*+1} a^{n-l^*+2} b_{n-l^*+1}}{(b_{n-l^*+1})^2}}$. Otherwise, when $|I_B| > l^*, n - |I_B| < n - l^*$, $\prod_{k=1}^{n-|I_B|} h_k$ will not include it.

Thus the item $g^{\frac{a^{q+1}}{b_{n-l^*+1}}}$ can be computed from $(\prod_{k=1}^{n-|I_B|} h_k)^{y_i}$ only when $|I_B| < l^*$. $\mathcal{B}$ also can calculate the remaining elements in $dk_B^{ID_i}$ as follows, because they do not involve a $g^{\frac{a^{q+1}}{b_{n-l^*+1}}}$ term. Thus, $\mathcal{B}$ computes $\{u_j^{-r_i-y_i}\}$ in user private keys:

$$\{u_j^{-r_i+\tilde{y}_i} \cdot g^{\frac{a^{2q-j}+\sum_{i=1,i\neq j}^{n} w_i a^{q-j+i+1} - \sum_{i=1}^{n} w_i w_j a^{i+2}}{b_q}} \cdot g^{\frac{a^{2q-(n-l^*)-j} - a^{q-(n-l^*)+1} w_j}{-b_q \gamma_{n-l^*+1}}}\},$$

And other parts in user private key $dk_B^{ID_i}$ are as follows:

$$g^{\alpha} w^{r_i} \cdot g^{x_i} \cdot (\prod_{k=1}^{n-|I_B|} h_k)^{r_i} \cdot (\prod_{k=1}^{n-|I_B|} h_k w)^{y_i} = g^{\tilde{\alpha}+\tilde{x_i}} \cdot w^{r_i+y_i} \cdot (\prod_{k=1}^{n-|I_B|} h_k)^{r_i} \cdot \Phi \cdot \Psi,$$

$$\{h_k^{r_i+y_i}\} = \{h_k^{r_i-\tilde{y}_i} \cdot g^{\frac{-\gamma_i(a^{q+k}+\sum_{t=1}^{n} \gamma_k w_t a^{t+k+1}) b_{n-l^*+1}}{(b_k)^2}} \cdot g^{\frac{\gamma_k a^{q-(n-l^*)+k} b_{n-l^*+1}}{(b_k)^2 \gamma_{n-l^*+1}}}\},$$

$$g^{r_i+y_i} = g^{r_i-\tilde{y}_i-a^{q-1}-\sum_{i=1}^{n} w_i a^i + \frac{a^{q-(n-l^*)-1}}{\gamma_{n-l^*+1}}}.$$

$\mathcal{B}$ sets user private key $dk_B^{ID_i} = (g^{\alpha} w^{r_i} \cdot g^{x_i} \cdot (\prod_{k=1}^{n-|I_B|} h_k)^{r_i} \cdot (\prod_{k=1}^{n-|I_B|} h_k w)^{y_i}, g^{r_i} \cdot g^{y_i}, \{u_j^{-r_i} \cdot u_j^{-y_i}\}_{j\in\{1,\ldots,|B|\}}, \{h_k^{r_i} \cdot h_k^{y_i}\}_{k\in\{n-|I_B|+1,\ldots,n\}})$ as above and sends them to $\mathcal{A}$.

– **Challenge.** When $\mathcal{A}$ decides that Phase 1 is over, it outputs two messages $M_0, M_1 \in \mathbb{G}$ on which it wishes to be challenged. $\mathcal{B}$ picks a random bit $b \in \{0, 1\}$ and responds with the challenge ciphertext

$$CT = (M_b \cdot T \cdot e(g, g^s)^{\tilde{\alpha}}, g^s, g^{\frac{sab_q}{b_1}}, \prod_{k=1}^{n-l^*} g^{\frac{s\gamma_k a^{k+1} b_{n-l^*+1}}{(b_k)^2}}, \{u_j^{t_j} v^{z_j}, g^{t_j}\}_{j\in\{1,\ldots,|A^*|\}})$$

where g^s and T are from the input tuple given to $\mathcal{B}$. First note that g^s, then $\mathcal{B}$ randomly chooses $z = sb_q, t_j, z_j \in \mathbb{Z}_p$, and calculates the ciphertext.

– **Query phase 2.** $\mathcal{A}$ issues queries not issued in Phase 1 and $\mathcal{B}$ responds as before. Specifically,
1. $\mathcal{A}$ issues private key queries that attribute sets $A_i \subseteq U$ satisfy $A^* \not\subseteq A_i$ and user identities ID_i, $i \in \{1, \ldots, q_1^{'}\}$.
2. $\mathcal{A}$ makes $q_2^{'}$ user private key queries that trusted high-level identity sets I_{B_j} satisfy $l^* > |I_{B_j}|$ under $A^* \subseteq B_j$, $j \in \{1, \ldots, q_2^{'}\}$.

– **Guess.** After the query phase 2, $\mathcal{A}$ outputs guess $b^{'}$ for the challenge bit. If $b^{'} = b$, $\mathcal{B}$ outputs 1, it claims that the challenge term is $T = e(g, g)^{sa^{q+1}}$. Otherwise, it outputs 0 meaning $T = R$ is random in $\mathbb{G}_T$.

When $T = R$, the ciphertext will give no information about the simulator's choice of b. In this case, we have $\Pr[b^{'} = b|T = R] = \Pr[b^{'} \neq b|T = R] = \frac{1}{2}$, beacuse the challenge ciphertext will contain only random numbers. Since the silumator outputs a guess $T^{'} = R$ when $b^{'} = b$, we then have $\Pr[T^{'} = T|T = R] = \frac{1}{2}$. When $T = e(g,g)^{sa^{q+1}}$, $\mathcal{A}$ can sees a valid encryotion of M_b. In this situation, the $\mathcal{A}$'s advantage is ϵ by definition. The probability is $\Pr[b^{'} = b|T = e(g,g)^{sa^{q+1}}] = \frac{1}{2} + \epsilon$. Since the simulator outputs a guess $T = e(g,g)^{sa^{q+1}}$ of T when $b^{'} = b$, we have $\Pr[T^{'} = T|T = e(g,g)^{sa^{q+1}}] = \frac{1}{2} + \epsilon$. Hence, by putting them all together, $\mathcal{B}$'s advantage in the above game is

$$
\begin{aligned}
&\Pr[T' = T] - \frac{1}{2} \\
=&\Pr[T' = T|T = e(g,g)^{sa^{q+1}}] \Pr[T = e(g,g)^{sa^{q+1}}] + \Pr[T' = T|T = R] \Pr[T = R] - \frac{1}{2} \\
=&\frac{1}{2} \Pr[T' = T|T = e(g,g)^{sa^{q+1}}] + \frac{1}{2} \Pr[T' = T|T = R] - \frac{1}{2} \\
=&\frac{1}{2}(\frac{1}{2} + \epsilon + \frac{1}{2}) - \frac{1}{2} \\
=&\frac{\epsilon}{2}
\end{aligned}
$$

Therefore, we obtain Theorem 1 and prove the security of our proposed scheme.

5 Discussion and Extensions

Efficiency. Based on the CP-ABE scheme in [23], both the size of the private key and the ciphertext in our scheme scale linearly with the number of attribute sets. For each trusted high-level user in the identity set I_B, the generation of a cooperative key ck_i^B requires $|I_B| - 1$ shared keys with other users. Specifically, each shared key needs $|I_B| - 1$ pairing operations. After the generation of a cooperative key, each trusted high-level user generates a user private key. Although the user private keys' generation adds extra $O(|I_B|^2)$ computation costs for users, the PKG concentrates on generating private keys for only a small number of high-level users, leading to a substantial improvement in the PKG key management.

Expressive Access Policy. In our KC-ABE scheme, the access policy is that the attribute set of ciphertext is the subset of the users' attribute set. Here we extend the access control to LSSS access policy (M, ρ) in [28]. When M_j, $\overrightarrow{v}$ are the vector corresponding to the jth row of M and a random vector, respectively, then $\lambda_j = \overrightarrow{v} \cdot M_j$ denotes the vector shares of the secret s. As a result, attribute elements in the ciphertext are represented as $u_j^{t_j} v^{\lambda_j}, g^{t_j}$, and the decryption procedure corresponds to the one in [28].

Chosen-Ciphertext Security. Our security definitions and proofs have been in the chosen-plaintext model. We remark that our construction can be extended

to the chosen-ciphertext model by applying the technique of one-time signature scheme in Canettu, Halevi, and Katz [4] by some simple alterations.

6 Conclusion

In this paper, we introduce a new concept called Key Cooperative ABE. It allows trusted users to generate user private keys for other users. This empowers users to decrypt ciphertexts using either the private key from the PKG or user private keys obtained from trusted users. Thus Key Cooperative ABE increases flexibility for users in obtaining decryption keys, solving the huge key management problem. Key Cooperative ABE avoids the introduction of additional attribute authorities and provides trust aggregation of delegation ability between multiple users in a cooperative group, making it more practical, lightweight, and extensible. We present a basic Key Cooperative ABE scheme that still poses several challenges, such as ensuring security against malicious users, developing more efficient constructions with shorter private keys and user private keys, and exploring its potential applications in scenarios.

References

1. Ashley, M.: The gnu privacy handbook (1999), `https://www.gnupg.org/gph/en/manual.html`
2. Bethencourt, J., Sahai, A., Waters, B.: Ciphertext-policy attribute-based encryption. In: IEEE Symposium on Security and Privacy 2007. pp. 321–334. IEEE Computer Society (2007)
3. Boneh, D., Franklin, M.K.: Identity based encryption from the weil pairing. IACR Cryptol. ePrint Arch. p. 90 (2001)
4. Canetti, R., Halevi, S., Katz, J.: Chosen-ciphertext security from identity-based encryption. In: Advances in Cryptology-EUROCRYPT 2004. pp. 207–222. Springer (2004)
5. Chase, M.: Multi-authority attribute based encryption. In: Vadhan, S.P. (ed.) TCC 2007. Lecture Notes in Computer Science, vol. 4392, pp. 515–534. Springer (2007)
6. Cui, H., Deng, R.H., Li, Y., Qin, B.: Server-aided revocable attribute-based encryption. In: Askoxylakis, I.G., Ioannidis, S., Katsikas, S.K., Meadows, C. (eds.) ESORICS 2016. LNCS, vol. 9879, pp. 570–587. Springer (2016)
7. Freire, E.S.V., Hofheinz, D., Kiltz, E., Paterson, K.G.: Non-interactive key exchange. In: Kurosawa, K., Hanaoka, G. (eds.) PKC 2013. Lecture Notes in Computer Science, vol. 7778, pp. 254–271. Springer (2013)
8. Garg, S., Hajiabadi, M., Mahmoody, M., Rahimi, A.: Registration-based encryption: Removing private-key generator from IBE. In: Beimel, A., Dziembowski, S. (eds.) TCC 2018. LNCS, vol. 11239, pp. 689–718. Springer (2018)
9. Goyal, V., Jain, A., Pandey, O., Sahai, A.: Bounded ciphertext policy attribute based encryption. In: Aceto, L., Damgård, I., Goldberg, L.A., Halldórsson, M.M., Ingólfsdóttir, A., Walukiewicz, I. (eds.) ICALP 2008. LNCS, vol. 5126, pp. 579–591. Springer (2008)
10. Hohenberger, S., Lu, G., Waters, B., Wu, D.J.: Registered attribute-based encryption. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023. LNCS, vol. 14006, pp. 511–542. Springer (2023)

11. Hohenberger, S., Waters, B.: Online/offline attribute-based encryption. In: Krawczyk, H. (ed.) PKC 2014. LNCS, vol. 8383, pp. 293–310. Springer (2014)
12. Horwitz, J., Lynn, B.: Toward hierarchical identity-based encryption. In: Knudsen, L.R. (ed.) EUROCRYPT 2002. LNCS, vol. 2332, pp. 466–481. Springer (2002)
13. Hur, J., Noh, D.K.: Attribute-based access control with efficient revocation in data outsourcing systems. IEEE Trans. Parallel Distributed Syst. **22**(7), 1214–1221 (2011)
14. Jiang, Y., Susilo, W., Mu, Y., Guo, F.: Ciphertext-policy attribute-based encryption with key-delegation abuse resistance. In: Information Security and Privacy: 21st Australasian Conference, ACISP 2016, Part I 21. pp. 477–494. Springer (2016)
15. Lai, J., Deng, R.H., Li, Y.: Fully secure cipertext-policy hiding CP-ABE. In: Bao, F., Weng, J. (eds.) ISPEC 2011. LNCS, vol. 6672, pp. 24–39. Springer (2011)
16. Lewko, A.B., Waters, B.: Decentralizing attribute-based encryption. IACR Cryptol. ePrint Arch. p. 351 (2010)
17. Li, J., Ren, K., Zhu, B., Wan, Z.: Privacy-aware attribute-based encryption with user accountability. In: Samarati, P., Yung, M., Martinelli, F., Ardagna, C.A. (eds.) ISC 2009. LNCS, vol. 5735, pp. 347–362. Springer (2009)
18. Lin, H., Cao, Z., Liang, X., Shao, J.: Secure threshold multi authority attribute based encryption without a central authority. Inf. Sci. **180**(13), 2618–2632 (2010)
19. Liu, Z., Cao, Z., Wong, D.S.: White-box traceable ciphertext-policy attribute-based encryption supporting any monotone access structures. IEEE Trans. Inf. Forensics Secur. **8**(1), 76–88 (2013)
20. Nishide, T., Yoneyama, K., Ohta, K.: Abe with partially hidden encryptor-specified access structure (2008)
21. Paterson, K.G., Srinivasan, S.: On the relations between non-interactive key distribution, identity-based encryption and trapdoor discrete log groups. Designs, Codes and Cryptography **52**, 219–241 (2009)
22. PGP, W.I.W.: Philip zimmermann
23. Rouselakis, Y., Waters, B.: Practical constructions and new proof methods for large universe attribute-based encryption. In: Sadeghi, A., Gligor, V.D., Yung, M. (eds.) CCS 2013. pp. 463–474. ACM (2013)
24. Sahai, A., Waters, B.: Fuzzy identity based encryption. IACR Cryptol. ePrint Arch. p. 86 (2004)
25. Sakai, R.: Cryptosystems based on pairing. SCIS 2000 (2000)
26. Shamir, A.: Identity-based cryptosystems and signature schemes. In: Blakley, G.R., Chaum, D. (eds.) CRYPTO 1984. LNCS, vol. 196, pp. 47–53. Springer (1984)
27. Wang, G., Liu, Q., Wu, J.: Hierarchical attribute-based encryption for fine-grained access control in cloud storage services. In: Al-Shaer, E., Keromytis, A.D., Shmatikov, V. (eds.) CCS 2010. pp. 735–737. ACM (2010)
28. Waters, B.: Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In: Catalano, D., Fazio, N., Gennaro, R., Nicolosi, A. (eds.) PKC 2011. LNCS, vol. 6571, pp. 53–70. Springer (2011)